



```
1 package com.fedex.app.tests.regressionTests;
2
3 import com.aventstack.extentreports.Status;
4
5
6 Run All
7
8 public class AssetPost extends Reporting {
9
10     private static final Logger log = LoggerFactory.getLogger(AssetPost.class);
11     String className = this.getClass().getSimpleName();
12     boolean checkTTLFlag;
13     String runFlag = "null";
14     String suiteName = PropertyConstants.TEST_TYPE_REGRESSION_TEST;
15     String fileName = "";
16     String tcName = "";
17     String readOktaToken, writeOktaToken, tcDescription = "";
18     String[] tokenValues = new String[2];
19     AssetDataUtils assetDataUtil = new AssetDataUtils();
20     private static final HttpClient client = HttpClient.newHttpClient();
21     BuilderUtility builderUtility = new BuilderUtility();
22     Queries query = new Queries();
23     Map<String, String> assetIdMap = new HashMap<String, String>();
24
25     @BeforeClass
26     public void before() throws Exception {
27         System.setProperty("https.proxyHost", "");
28         System.setProperty("https.proxyPort", "00");
29         log.info("in Before method");
30         String appName = "";
31         log.info(className + ":");
32         super.startReport(suiteName + "/" + className);
33         try {
34             appName = RunProperties.getInstance().getAppName();
35             fileName = appName + " " + suiteName;
36         }
37     }
38 }
```

```

58 String appName = "";
59 log.info(className + ":");
60 super.startReport(suiteName + "/" + className);
61 try {
62     appName = RunProperties.getInstance().getAppName();
63     fileName = appName + "_" + suiteName;
64     log.info("in Before method fileName: " + fileName);
65 } catch (IOException e) {
66     e.printStackTrace();
67     test.fail("Unable to get the file name, exception thrown: " + e.toString());
68 }
69
70 // *** get token properties
71 tokenValues = RetrieveEndpoints.getTokenProperties(fileName, className);
72 // *** get token
73 writeOktaToken = fetchToken(tokenValues[1], PropertyConstants.ASSET_WRITE_SCOPE);
74 // Fetching token value
75 readOktaToken = fetchToken(tokenValues[1], PropertyConstants.ASSET_READ_SCOPE);
76 assetDataUtil.loadAssetProps();
77 log.info("in Before method end");
78 }
79
80 @BeforeMethod
81 protected void startRepo(Method m) throws IOException {
82     log.info("in startRepo");
83
84
85     tcName = m.getName();
86     log.info(" " + tcName + ":");
87     if (tcName.equals("TC_01") && runFlag.equalsIgnoreCase("Yes")) {
88         test = extent.createTest("Actor: In");
89     }
90 }

```

# OrchestratorOfXERegressionTest.java

AssetPost.java

```
log.info("TC_01")
if (tcName.equals("TC_01") && runFlag.equalsIgnoreCase("Yes")) {
    test = extent.createTest("Actuator Info");
}
```

}

```
private String fetchToken(String tokenValue, String scope) {
    try {
```

```
        URI uri = URI.create(tokenValue + scope);
        HttpRequest req = HttpRequest.newBuilder().uri(uri).GET().build();
```

```
        HttpResponse<String> response = client.send(req, HttpResponse.BodyHandlers.ofString());
        return response.body();
    } else {
```

```
        log.error("Failed to fetch token");
        return null;
    }
}
```

```
} catch (IOException | InterruptedException e) {
    log.error("Error fetching token", e);
    return null;
}
```

}

```
@DataProvider(name = "SuccessTestCasesScenario")
public Object[][] SuccessTestCasesScenario() {
```

```
    return new Object[][] { { builderUtility.createAssetWithChanges(request -> request.getHeader(), TestCaseName
```

```
        { builderUtility.createAssetWithChanges(request -> request.getHeader().setApiVersion(PropertyCore
```

```
        { builderUtility.createAssetWithChanges(request -> request.getHeader().setApiVersion(PropertyCore
```

```
        { builderUtility.createAssetWithChanges(request -> request.getHeader().setSourceRequestId(Prop
```

 \*AssetPost.java 

```

9 { builderUtility.createAssetWithChanges(request -> request.getAsset()).setExpirationTimestamp(assetDataUtil.ge
0
1 { builderUtility.createAssetWithChanges(request -> request.getAsset()).setUnitNumber(assetDataUtil.ge
2 { builderUtility.createAssetWithChanges(request -> request.getAsset()).setUnitNumber(assetDataUtil.ge
3
4 { builderUtility.createAssetWithChanges(request -> request.getAsset()).setAssetStartDate(assetDataUtil.ge
5 { builderUtility.createAssetWithChanges(request -> request.getAsset()).setAssetStartDate(assetDataUtil.ge
6 { builderUtility.createAssetWithChanges(request -> request.getAsset()).setAssetStartDate(assetDataUtil.ge
7
8 { builderUtility.createAssetWithChanges(request -> request.getAsset()).
9     setAssetEndDate(PropertyConstants.NULL), TestCaseNameConstants.TC_080),
0 { builderUtility.createAssetWithChanges(request -> request.getAsset()).
1     setAssetEndDate(PropertyConstants.EMPTY), TestCaseNameConstants.TC_081),
2 { builderUtility.createAssetWithChanges(request -> request.getAsset()).setAssetEndDate(assetDataUtil.ge
3 { builderUtility.createAssetWithChanges(request -> request.getAsset()).setAssetEndDate(assetDataUtil.ge
4 { builderUtility.createAssetWithChanges(request -> request.getAsset()).setAssetEndDate(assetDataUtil.ge
5
6
7 { builderUtility.createAssetWithChanges(request -> {
8     request.getAsset().setAssetStartDate(PropertyConstants.NULL);
9     request.getAsset().setAssetEndDate(PropertyConstants.NULL);
0
1     }, TestCaseNameConstants.TC_090),
2 { builderUtility.createAssetWithChanges(request -> {
3     request.getAsset().setAssetStartDate(PropertyConstants.EMPTY);
4     request.getAsset().setAssetEndDate(PropertyConstants.EMPTY);
5
6     }, TestCaseNameConstants.TC_091),
7
8

```

```
79 { builderUtility.createAssetWithChanges(request -> request.getHeader().setWeightRange(PropertyConstants.EMPTY))
80 { builderUtility.createAssetWithChanges(request -> request.getHeader().setWeightRange(PropertyConstants.EMPTY))
81 { builderUtility.createAssetWithChanges(request -> request.getHeader().setWeightRange(PropertyConstants.SPACE))
82 { builderUtility.createAssetWithChanges(request -> request.getHeader().setWeightRange(PropertyConstants.SPECIAL
83
84 { builderUtility.createAssetWithChanges(request -> request.getHeader().setWeightClass(PropertyConstants.NULL))
85 { builderUtility.createAssetWithChanges(request -> request.getHeader().setWeightClass(PropertyConstants.EMPTY))
86 { builderUtility.createAssetWithChanges(request -> request.getHeader().setWeightClass(PropertyConstants.EMPTY))
87 { builderUtility.createAssetWithChanges(request -> request.getHeader().setWeightClass(PropertyConstants.SPACE))
88 { builderUtility.createAssetWithChanges(request -> request.getHeader().setWeightClass(PropertyConstants.SPECIAL
89
90
91
92 builder(name = "ErrorTestCaseScenario")
93 object[] [] ErrorTestScenario() {
94
95     new Object[] [] {
96         { builderUtility.createAssetWithChanges(request -> request.getHeader(null)),
97           TestCaseNameConstants.TC_002, ErrorMessage.OBJECT_IS_MISSING),
98         { builderUtility.createAssetWithChanges(request -> request.setAsset(null)), TestCaseNameConstants.TC_003, ErrorMessage
99
100 { builderUtility.createAssetWithChanges(request -> request.getHeader().setApiVersion(PropertyConstants.NULL)), Test
101 { builderUtility.createAssetWithChanges(request -> request.getHeader().setApiVersion(PropertyConstants.EMPTY)), Test
102 { builderUtility.createAssetWithChanges(request -> request.getHeader().setApiVersion(PropertyConstants.SPACE)), Test
103 { builderUtility.createAssetWithChanges(request -> request.getHeader().setApiVersion(PropertyConstants.SIZE_VALID)
104
105 { builderUtility.createAssetWithChanges(request -> request.getHeader().setSourceRequestId(PropertyConstants.NULL)
106 { builderUtility.createAssetWithChanges(request -> request.getHeader().setSourceRequestId(PropertyConstants.EMPTY)
107 { builderUtility.createAssetWithChanges(request -> request.getHeader().setSourceRequestId(PropertyConstants.SPACE)
108 { builderUtility.createAssetWithChanges(request -> request.getHeader().setSourceRequestId(PropertyConstants.SIZE
109
110 { builderUtility.createAssetWithChanges(request -> request.getHeader().setEffectiveTimestamp(PropertyConstants.SPACE
111 { builderUtility.createAssetWithChanges(request -> request.getHeader().setEffectiveTimestamp(PropertyConstants.SPECIAL
```

}), TestCaseNameConstants.TC\_040),

};

@Test(dataProvider = "SuccessTestCaseScenario")  
Run | Debug

public void SuccessTestCaseScenario(AssetRequestPostDTO assetRequestPostDTO, String testCaseID) throws  
fetchTestCaseDescription(testCaseID);  
successTestCase(testCaseID, assetRequestPostDTO);  
}

@Test(dataProvider = "ErrorTestCaseScenario")  
Run | Debug

public void ErrorTestCaseScenario(AssetRequestPostDTO assetRequestPostDTO, String testCaseID, String erro  
fetchTestCaseDescription(testCaseID);  
ErrorTestCase(assetRequestPostDTO, ErrorMessage.SYNTAX2, errorMsg);  
}

@Test(dataProvider = "SuccessTestCaseScenarioTTL")  
Run | Debug

public void SuccessTestCaseScenarioTTL(AssetRequestPostDTO assetRequestPostDTO, String testCaseID) throws  
fetchTestCaseDescription(testCaseID);  
checkTTLFlag = true;  
successTestCase(testCaseID, assetRequestPostDTO);  
}

private void fetchTestCaseDescription(String testCaseID) throws IOException {



Access to int  
authenticate

Authenticate

Writable

Smart Inset

```

    }
    test.log(Status.INFO, MarkupHelper.createLabel(String.valueOf(tcDescription), ExtentColor.PURPLE));
}

public void successTestCase(String testCaseId, AssetRequestPostDTO assetRequestPostDTO) throws Exception {
    test.info("Input Request Created:");
    test.info(assetRequestPostDTO.toString());

    //Asset Post
    Response response = postRequest(assetRequestPostDTO, writeOktaToken);
    test.info("Expected Response Code: 200 ");
    test.pass("Response Status Code " + response.getStatusCode());
    if (response.getStatusCode() == 200) {
        AssetResponse assetResponse = response.as(AssetResponse.class);
        String assetId = assetResponse.getAssets().get(0).getAssetId();
        saveToMap(assetId);
        test.pass("Response Status Validation Passed: " + 200);
        Thread.sleep(20000);
        String Asset_PostQuery = query.asset_Post_Query(assetId);
        List<String> fields = ValidationFields.assetPost_DB_Fields();
        // ***get the result from DB
        List<String> getResultDB = DbConnect.getResultSetFor(Asset_PostQuery, fields, fileName, testCaseId);

        AssetPostDTO asset = assetRequestPostDTO.getAsset();
        TransportationDTO transportation = asset.getTransportation();

        String effectiveTimestamp=asset.getEffectiveTimestamp();
        String expirationTimestamp=asset.getExpirationTimestamp();
        if (expirationTimestamp==null || expirationTimestamp.isEmpty())
            expirationTimestamp = "5555-10-31T23:59:59Z";

        if (effectiveTimestamp==null || effectiveTimestamp.isEmpty())

```

```

String effectiveTimestamp=asset.getEffectiveTimestamp();
String expirationTimestamp=asset.getExpirationTimestamp();
if (expirationTimestamp==null || expirationTimestamp.isEmpty())
    expirationTimestamp = "9999-12-31T23:59:59Z";

if (effectiveTimestamp==null || effectiveTimestamp.isEmpty())
{
    effectiveTimestamp =DateUtility.convertTimestampToString(PropertyConstants.TIMESTAMP_PATTERN,DateUtili
}

asset=modifyAssetFieldToValidateDb(asset);

String[] inputFieldValues = { effectiveTimestamp, expirationTimestamp,
    asset.getTypeCode(), asset.getStatusCode(), asset.getAssetStartDate(), asset.getAssetEndDate(),
    asset.getCategoryCode(), asset.getSubTypeCode(), asset.getOwnedByParty(), asset.getVIN(),
    asset.getDomicileLocationAliasNumber(), asset.getWeightUnit(),
    transportation.getGrossVehicleWeightRating(),
    asset.getWeightClass(), asset.getWeightRange() };

List<String> resFields = ValidationFields.assetPost_Response_Fields();
boolean testResult = TestResultValidation.testValidation_withDB(response, inputFieldValues, getResultDB,
    resFields);
if (testResult)
{
    if (checkTTIFlag) {
        String AssetPostTTIQuery = query.AssetTTI_Query(assetId);
        List<String> fieldTTI = ValidationFields.IdentifierTTI_DB_Fields();

        Timestamp convertedExpirationTstp = Miscellaneous.convertToTimestamp(PropertyConstants.TIME
        Timestamp refYear1 = Miscellaneous.convertToTimestamp(PropertyConstants.TIME
        if (convertedExpirationTstp.after(refYear1)) || (convertedExpirationTstp.equals
            test.info("Expiration is greater than or equal to 01/01/2031, do not set TI
        } else {

```

```

Timestamp convertedExpirationTstp = Miscellaneous.convertToTimestamp(PropertyConstants
Timestamp refYear1 = Miscellaneous.convertToTimestamp(PropertyConstants.TIMESTAMP_PART
if (convertedExpirationTstp.after(refYear1) || (convertedExpirationTstp.equals(refYear
    test.info("ExpDate is greater than or equal to 01/01/2031, do not set TTL");
} else {
    long getResultDBTTLValue = DbConnect.getResultsetForIdentifierTTL(AssetPostTTLQuer
        fileName, testCaseID);
    test.info("TTL value retrieved from database: " + getResultDBTTLValue);
    long ttlVal = Miscellaneous.calculateTTL(expirationTimestamp);
    test.info("ExpDate is less than 01/01/2031, set TTL");
    test.info("Calculated ttl value before adding 10:30 hrs: " + ttlVal);
    long ttlVall = Miscellaneous.calculateTTL(expirationTimestamp) + 19803L;
    test.info("Calculated ttl value after adding 10:30 hrs: " + ttlVall);
    test.info("Difference between Calculated ttl and db ttl value:"
        + (ttlVall - getResultDBTTLValue));
}

if ((getResultDBTTLValue - ttlVall) <= 10) {
    test.pass("TTL Comparison with Database Passed");
} else {
    test.fail("TTL Comparison with Database Failed");
}
}

checkTTLFlag=false;
}

successTest(response);
}

failedTest(response);
}

```

```

427 test.fail("TTL Comparison with Database Failed");
428 }
429
430     }
431     checkTTLFlag=false;
432     }
433     successTest(response);
434 }
435 else
436     failedTest(response);
437 } else
438     failedTest(response);
439 }
440
441 public Response postRequest(AssetRequestPostDTO request,String token) {
442
443     Response response = null;
444     String getEndPointUrl = RetrieveEndpoints.getEndPointUrl("assetpost", fileName, className);
445     try {
446         RequestSpecification specification = RestAssured.with();
447         specification.given().contentType("application/json");
448         specification.header(HttpHeaders.AUTHORIZATION, token);
449         log.info("Post Request " + request);
450         log.info("url ~ " + getEndPointUrl);
451         response = specification.body(request).when().post(getEndPointUrl);
452
453         log.info("Post response " + response.asString());
454     } catch (Exception e) {
455         e.printStackTrace();
456         log.error(e.getMessage());
457     }
458     return response;
459 }

```

```

456         log.error(e.getMessage());
457     }
458     return response;
459 }
460
461 private void saveToMap(String assetId) {
462
463     int count = assetIdMap.size();
464     String key = "NPO_" + String.valueOf(count);
465     assetIdMap.put(key, assetId);
466 }
467
468 private AssetPostDTO modifyAssetFieldToValidateDb(AssetPostDTO asset) {
469
470     if(asset.getAssetStartDate()==null )
471         asset.setAssetStartDate("");
472     if(asset.getAssetEndDate()==null )
473         asset.setAssetEndDate("");
474     if(asset.getUnitNumber()!=null && (asset.getUnitNumber().isEmpty() || asset.getUnitNumber().equals("NA")))
475         asset.setUnitNumber("");
476     if(asset.getVIN()!=null && (asset.getVIN().equals("null") || asset.getVIN().equals("NA")))
477         asset.setVIN("");
478     return asset;
479 }
480
481 public void successTest(Response response) {
482
483     test.pass("Comparison With Database Passed");
484     log.info("Test Status is UP");
485     test.log(Status.PASS, MarkupHelper.createLabel("Test Status : PASS", ExtentColor.GREEN));
486 }
487

```

```
493 Assert.fail("Test Failed");
494 }
495
```

```
496 public void ErrorTestCase(AssetRequestPostDTO assetRequestPostDTO, String errorMsg, String errorValue)
497
```

```
498     test.info("Input Request created:");
```

```
499     test.info(assetRequestPostDTO.toString());
```

```
500     Response response = postRequest(assetRequestPostDTO, writeAccessToken);
```

```
501     test.info("Response:");
```

```
502     test.info(response.asString());
```

```
503     test.info("Expected Response Code: 400 ");
```

```
504     AtomicBoolean isValueMatch = new AtomicBoolean(false);
```

```
505     if (response.getStatusCode() == 400) {
```

```
506         AssetResponse assetResponse = response.as(AssetResponse.class);
```

```
507
508         List<ResponseMessage> responseMessages = assetResponse.getHeader().getResponseMessages();
```

```
509         responseMessages.forEach(resMsg -> {
```

```
510             if (resMsg.getMessage().equalsIgnoreCase(errorMessage)) {
```

```
511                 if (resMsg.getParameters().stream()
```

```
512                     .anyMatch(param -> param.getValue().equalsIgnoreCase(errorValue)))
```

```
513                     isValueMatch.set(true);
```

```
514             }
```

```
515         });
```

```
516         if (isValueMatch.get())
```

```
517             updateSuccessStatus(response);
```

```
518         else
```

```
519             updateErrorStatus(response);
```

```
520     } else
```

```
        .thenReturn(param -> param.getValue().equalsIgnoreCase(errorValue))
        isValueMatch.set(true);
    }
}
```

```
    });
}
```

```
    if (isValueMatch.get())
```

```
        updateSuccessStatus(response);
```

```
    else
```

```
        updateErrorStatus(response);
```

```
    } else
```

```
        updateErrorStatus(response);
```

```
    }
```

```
    public void updateSuccessStatus(Response response) {
```

```
        test.pass("Response Status Code " + response.getStatusCode());
```

```
        test.pass("Response Error Message Validation Passed");
```

```
        log.info("Test Status is UP");
```

```
        test.log(Status.PASS, MarkupHelper.createLabel("Test Status : PASS", ExtentColor.GREEN));
```

```
    }
```

```
    public void updateErrorStatus(Response response) {
```

```
        test.fail("Response Status Code : " + response.getStatusCode());
```

```
        test.fail("Response Error Message Validation Failed");
```

```
        log.info("Test Status is DOWN");
```

```
        test.log(Status.FAIL, MarkupHelper.createLabel("Test Status : FAIL", ExtentColor.RED));
```

```
        Assert.fail("Test Failed");
```

```
    }
```

```
@AfterClass
```

```
public void after() throws Exception {
```

```

534
535 test.fail("Response Status Code : " + response.getStatusCode());
536 test.fail("Response Error Message Validation Failed");
537 log.info("Test Status is DOWN");
538 test.log(Status.FAIL, MarkupHelper.createLabel("Test Status : FAIL", ExtentColor
539 Assert.fail("Test Failed"));
540 }
541
542 @AfterClass
543 public void after() throws Exception {
544
545     try {
546         Path path = Paths.get(PropertyConstants.ASSET_POST_PROP_FILE);
547         BufferedWriter writer = Files.newBufferedWriter(path);
548         for (Map.Entry<String, String> map : assetIdMap.entrySet()) {
549             writer.write(map.getKey() + "=" + map.getValue());
550             writer.newLine();
551         }
552         writer.close();
553     }
554 } catch (Exception e) {
555     e.printStackTrace();
556 }
557 }
558
559 @AfterSuite
560 public void endReportSuite() {
561     extent.flush();
562 }
563 }

```



OrchestratorFXERegressionTest.java \*AssetPostjava un\_endPoints\_AssetModule.properties

1 ## URI Bases

2 3 # Release 4 Colo environment URL

4 clwdev111=https://asset-services-assetapi-dev.app.clwdev1.paas.fedex.com/

5 clwdev114=https://asset-services-assetapi-core-rel.app.clwdev1.paas.fedex.com/

6 clwdev212=https://asset-services-assetapi-dev.app.clwdev2.paas.fedex.com/

7 clwdev214=https://asset-services-assetapi-core-rel.app.clwdev2.paas.fedex.com/

8 clwdev313=https://asset-services-assetapi-dev.app.clwdev3.paas.fedex.com/

9 clwdev314=https://asset-services-assetapi-core-rel.app.clwdev3.paas.fedex.com/

10

11

12 clwdev111.assetActuatorClw=https://asset-services-assetapi-dev.app.clwdev1.paas.fedex.com/actuator/info

13 clwdev114.assetActuatorClw=https://asset-services-assetapi-core-rel.app.clwdev1.paas.fedex.com/actuator/info

14 clwdev212.assetActuatorClw=https://asset-services-assetapi-dev.app.clwdev2.paas.fedex.com/actuator/info

15 clwdev214.assetActuatorClw=https://asset-services-assetapi-core-rel.app.clwdev2.paas.fedex.com/actuator/info

16 clwdev313.assetActuatorClw=https://asset-services-assetapi-dev.app.clwdev3.paas.fedex.com/actuator/info

17 clwdev314.assetActuatorClw=https://asset-services-assetapi-core-rel.app.clwdev3.paas.fedex.com/actuator/info

18

19 cldev111=https://asset-services-assetapi-rel.app.cldev1.paas.fedex.com/

20 cldev114=https://asset-services-assetapi-core-rel.app.cldev1-az1.paas.fedex.com/

21 cldev212=https://asset-services-assetapi-dev.app.cldev1-az2.paas.fedex.com/

22 cldev214=https://asset-services-assetapi-core-rel.app.cldev1-az2.paas.fedex.com/

23 cldev313=https://asset-services-assetapi-dev.app.cldev1-az3.paas.fedex.com/

24 cldev314=https://ephemeral-asset-api-ib-las.test.cloud.fedex.com:8081/

25 cldev314=https://asset-services-assetapi-core-rel.app.cldev1-az3.paas.fedex.com/

26

27 cldev111.assetActuatorClw=https://asset-services-assetapi-rel.app.cldev1-az1.paas.fedex.com/actuator/info

28 cldev114.assetActuatorClw=https://asset-services-assetapi-core-rel.app.cldev1-az1.paas.fedex.com/actuator/info

29 cldev212.assetActuatorClw=https://asset-services-assetapi-dev.app.cldev1-az2.paas.fedex.com/actuator/info

30 cldev214.assetActuatorClw=https://asset-services-assetapi-core-rel.app.cldev1-az2.paas.fedex.com/actuator/info

31 cldev313.assetActuatorClw=https://asset-services-assetapi-dev.app.cldev1-az3.paas.fedex.com/actuator/info

32 cldev313.assetActuatorClw=http://ephemeral-asset-api-ib-las.test.cloud.fedex.com:8081/actuator/info

33 cldev314.assetActuatorClw=https://asset-services-assetapi-core-rel.app.cldev1-az3.paas.fedex.com/actuator/info

Writable

Insert

1:1:0

OrchestratorFXERegressionTest.java

\*AssetPost.java

uri\_endPoints\_AssetModule.properties

```
34
35
36
37
38 #clwdevl1=https://asset-services-assetapi-core-dev.app.clwdevl.paas.fedex.com/assets/
39
40 #COLO
41 colol4=https://asset-cloud-gateway.app.clwdevl.paas.fedex.com/
42
43 colol4.assetActuatorEdc=https://asset-services-assetapi-core-dev.app.clwdevl.paas.fedex.com/actuator/info
44
45 ## Below are the URI Details
46
47 #assets Service
48 asset.post=v1/assets
49 asset.post.vn=v2/assets
50 asset.put=v1/assets
51 asset.put.vn=v2/assets
52 asset.get=v1/assets/
53 asset.get.v2=v2/assets/
54 asset.getHistory=v1/assets/history
55 asset.getHistory.v2=v2/assets/history
56 asset.findBy=v1/assets/findBy
57 asset.findBy.v2=v2/assets/findBy
58 asset.getAssetByVin=v1/assets/getAssetByVIN
59 asset.getAssetByVin.v2=v2/assets/getAssetByVIN
60 asset.getAssets.v2=v2/assets/getAssets
61 asset.rollback=v1/assets/rollback/
62 asset.rollback.v2=v2/assets/rollback/
63 asset.deleteById=v1/assets/
64 asset.deleteById.v2=v2/assets/
65 actuator.health=actuator/health
66 actuator.info=actuator/info
```